

STA 445 S26 Assignment 4

2026-03-06

Load all the packages you will use in this chunk. Notice, I have set `include = FALSE` because I don't want to see you loading packages in the output.

Problem 1

For the following regular expression, explain in words what it matches on. Then add test strings to demonstrate that it in fact does match on the pattern you claim it does. Do at least 4 tests. Make sure that your test set of strings has several examples that match as well as several that do not. Make sure to remove the `eval=FALSE` so that the code and output shows in your pdf.

- a. This regular expression matches: *This regex detects specifically lowercase a anywhere within the string*

```
strings <- c("abcdef", "iiiiiaiiii", "AAA", "bcd")
data.frame( string = strings ) %>%
  mutate( result = str_detect(string, 'a') )
```

```
##      string result
## 1      abcdef  TRUE
## 2 iiiiiaiiii  TRUE
## 3         AAA FALSE
## 4         bcd FALSE
```

- b. This regular expression matches: *This regex detects specifically the lowercase string ab anywhere within the string*

```
strings <- c("ab", "123ab45", "a b", "AB", "ba")
data.frame( string = strings ) %>%
  mutate( result = str_detect(string, 'ab') )
```

```
##      string result
## 1         ab  TRUE
## 2 123ab45  TRUE
## 3      a b FALSE
## 4        AB FALSE
## 5        ba FALSE
```

- c. This regular expression matches: *Any string with either lowercase b or a*

```
strings <- c("B","wwaww","b","123","cde")
data.frame( string = strings ) %>%
  mutate( result = str_detect(string, '[ab]') )
```

```
##   string result
## 1      B  FALSE
## 2 wwaww   TRUE
## 3      b   TRUE
## 4    123  FALSE
## 5     cde  FALSE
```

d. This regular expression matches: *Any string where the first character is a lowercase a or b*

```
strings <- c("1a","A123","ab","b23")
data.frame( string = strings ) %>%
  mutate( result = str_detect(string, '^[ab]') )
```

```
##   string result
## 1     1a  FALSE
## 2    A123  FALSE
## 3     ab   TRUE
## 4    b23   TRUE
```

e. This regular expression matches: *Strings where there is a substring consisting of a digit, a white space character, and then an a or A*

```
strings <- c("9
A","5 1 a", "aiwoaoit6 A","6 a wawetwet", "oi6 Awe","6 w a","6  a")
data.frame( string = strings ) %>%
  mutate( result = str_detect(string, '\\d+\\s[aA]') )
```

```
##           string result
## 1      9\nA   TRUE
## 2     5 1 a   TRUE
## 3 aiwoaoit6 A   TRUE
## 4 6 a wawetwet   TRUE
## 5     oi6 Awe   TRUE
## 6      6 w a  FALSE
## 7      6  a  FALSE
```

f. This regular expression matches: *Strings where there is a substring of a digit and then an A or a, with any number of spaces (including 0) between them*

```
strings <- c("6a","6      A","4    w a","www6awww","2ba")
data.frame( string = strings ) %>%
  mutate( result = str_detect(string, '\\d+\\s*[aA]') )
```

```
##      string result
## 1      6a    TRUE
## 2 6      A    TRUE
## 3  4     w a  FALSE
## 4  www6aww   TRUE
## 5      2ba  FALSE
```

g. This regular expression matches: *Any string (any string with 0 or more .s)*

```
strings <- c("", "....", "aa.aa", "aa")
data.frame( string = strings ) %>%
  mutate( result = str_detect(string, '.*') )
```

```
##      string result
## 1              TRUE
## 2      ....    TRUE
## 3  aa.aa    TRUE
## 4      aa    TRUE
```

h. This regular expression matches: *Strings where the first two characters are alphanumeric and the following three are “bar”*

```
strings <- c("mmbar", "12bar", "123bar", "abbar1234567", " bar")
data.frame( string = strings ) %>%
  mutate( result = str_detect(string, '^\\w{2}bar') )
```

```
##      string result
## 1      mmbar    TRUE
## 2      12bar    TRUE
## 3     123bar  FALSE
## 4 abbar1234567  TRUE
## 5       bar   FALSE
```

i. This regular expression matches: *Strings which either have “foo.bar” as a substring or start with two alphanumeric characters followed by “bar”*

```
strings <- c("foo.bar", "aabar", "12foo.bar34", "12aabar", "foobar")
data.frame( string = strings ) %>%
  mutate( result = str_detect(string, '(foo\\.bar)|(^\\w{2}bar)') )
```

```
##      string result
## 1    foo.bar    TRUE
## 2     aabar    TRUE
## 3 12foo.bar34    TRUE
## 4    12aabar  FALSE
## 5     foobar  FALSE
```

Problem 2

The following file names were used in a camera trap study. The S number represents the site, P is the plot within a site, C is the camera number within the plot, the first string of numbers is the YearMonthDay and the second string of numbers is the HourMinuteSecond.

```
file.names <- c( 'S123.P2.C10_20120621_213422.jpg',
                 'S10.P1.C1_20120622_050148.jpg',
                 'S187.P2.C2_20120702_023501.jpg')
```

Produce a data frame with columns corresponding to the site, plot, camera, year, month, day, hour, minute, and second for these three file names. So we want to produce code that will create the data frame:

Site	Plot	Camera	Year	Month	Day	Hour	Minute	Second
S123	P2	C10	2012	06	21	21	34	22
S10	P1	C1	2012	06	22	05	01	48
S187	P2	C2	2012	07	02	02	35	01

```
file.df <- data.frame(original_name=file.names)
cols <- c('Site','Plot','Camera','Date','Time','Ext')
split.df <- file.df %>%
  separate(col=original_name,into = cols,sep="[_.]")
final.df <- split.df %>% mutate(
  Year = str_sub(Date,1,4),
  Month = str_sub(Date,5,6),
  Day = str_sub(Date,7,8),
  Hour = str_sub(Time,1,2),
  Minute = str_sub(Time,3,4),
  Second = str_sub(Time,5,6),
  Date = NULL,
  Time = NULL,
  Ext = NULL)
final.df
```

##	Site	Plot	Camera	Year	Month	Day	Hour	Minute	Second
## 1	S123	P2	C10	2012	06	21	21	34	22
## 2	S10	P1	C1	2012	06	22	05	01	48
## 3	S187	P2	C2	2012	07	02	02	35	01

3. The full text from Lincoln's Gettysburg Address is given below. Calculate the mean word length
Note: consider 'battle-field' as one word with 11 letters).

```
Gettysburg <- 'Four score and seven years ago our fathers brought forth on this continent, a new
```

```
wordlist <- Gettysburg %>%
  str_remove_all(pattern="-- ") %>%
  str_remove_all(pattern="-") %>%
```

```
str_remove_all(pattern="\\.") %>%  
str_remove_all(pattern="," ) %>%  
str_split(pattern = "\\s+")  
wordlist <- wordlist[[1]]  
mean(str_length(wordlist))
```

```
## [1] 4.239852
```